

Executive Summary

This **AI Job Hunter** is a 100% free, public-GitHub-repo system to continuously discover, score and manage job openings without any cost. It uses **GitHub Actions** as the scheduler (triggering every ~15 minutes), **Supabase** as the backend database/storage, and open-source tools and models for automation and AI. It integrates only *permitted* APIs and feeds (no LinkedIn scraping or TOS violations). Key components include:

- **Data Sources:** Public job feeds and APIs (e.g. [Arbeittnow](#), RemoteOK RSS/JSON, WeWorkRemotely RSS, USAJOBS API, etc.) and company career endpoints (e.g. Greenhouse job board APIs).
- **Crawling & Scheduling:** GitHub Actions workflows (cron) fetch jobs via these APIs/feeds. Optionally, self-hosted automation tools (n8n, ActivePieces, Huginn, etc.) can orchestrate flows, though GH Actions suffices for scheduling. No paid or external services are required.
- **Data Pipeline:** Crawlers collect new jobs, deduplicate by unique identifiers or content similarity, and store them in Supabase tables. An AI scoring engine ranks jobs by match to the user's profile (skills, location, seniority).
- **AI Resume Tailoring:** We leverage free/open-source NLP models (e.g. [nakamoto-yama/t5-resume-generation](#) [28†L203-L212]) to generate a draft resume (in JSON or text) tailored to each job description. This JSON is then merged into a LaTeX resume template (one per target role, e.g. "Platform Engineer", "Backend Developer", etc.), producing a personalized PDF/DOCX.
- **Storage & UI:** All jobs and tailored resumes are stored in Supabase (a free Postgres-based BaaS [37†L15-L19]). A Next.js dashboard (publicly hosted) lets the user review jobs once a day, view scores, and click "Apply" links. No email is sent – everything is visible in the Supabase dashboard (as preferred).
- **Cost & Open Source:** All components are free and self-hosted. Supabase and GitHub Actions free tiers cover usage. We only use OSS libraries and models (ActivePieces, n8n, Hugging Face models, etc.), and avoid paid APIs or proprietary services.

In the sections below we detail **(1)** Objectives & Scope, **(2)** Data Sources and Access Methods, **(3)** Tools & Technology Inventory (with pros/cons), **(4)** Architecture and Data Flow (with diagrams), **(5)** Database Schema & GitHub Actions examples, **(6)** Crawler and AI pipeline design, **(7)** Implementation steps (incl. code snippets and LaTeX templates), and **(8)** Scaling, learning, and compliance notes. All assertions are backed by open-source documentation and official sources.

1. Objectives and Scope

- **Goal:** Build an "*AI Job Hunter*" that continuously scans relevant job postings, scores them by match, auto-generates tailored resumes, and makes applying as easy as one click (to start the application process).
- **Public & Free:** The code must be in a public GitHub repo and incur *zero monetary cost*. We rely only on free/open-source tools and APIs. Any cloud usage (Supabase, GitHub Actions, Next.js hosting) is within their free tiers.
- **Data Privacy & Compliance:** No LinkedIn scraping, no violating site TOS. Use only official feeds/APIs (RSS, JSON, sanctioned job board APIs). Securely handle any credentials (stored as GitHub Secrets) and follow GDPR/ATS privacy guidelines.

- **Scope:** We will target international/remote engineering jobs. Unspecified details (e.g. exact companies or keywords) are made configurable. The system is modular – new sources, models or resume templates can be added as needed.

Key Features:

- **Job Discovery:** Poll multiple sources (see next section), retrieve new listings.
- **Deduplication:** Identify and ignore duplicate postings (by URL, title+company, or content-hash).
- **Scoring/Ranking:** For each job, compute a “match score” based on your profile (skills, seniority, location preferences, etc.). We will discuss algorithms below.
- **Resume Tailoring:** For top-matched jobs, automatically generate a resume draft. A LaTeX template is filled using AI-generated content.
- **Dashboard:** Supabase-backed UI listing jobs, scores, and “Apply” links. A review status can be set (Applied / Rejected / Ignored) to train the system over time.
- **Learning Loop:** The system adapts to your actions (e.g. jobs you apply to or reject) to improve future recommendations (described in Migration notes).

The deliverables cover **strategy, design, and implementation** for all these components, with code examples and references to OSS documentation.

2. Data Sources & Access Methods

We gather jobs from **(a) Job Aggregators/APIs** and **(b) Company Career Pages/ATS**. Only officially supported methods are used (no HTML scraping of blocked sites).

2.1 Job Aggregators & APIs

- **Arbeitnow (Free Job Board API):** A free API aggregating jobs (mainly EU/remote) from major Applicant Tracking Systems (Greenhouse, SmartRecruiters, Join.com, Teamtailor, Recrutee, Comeet, etc.) [22†L68-L77] . Endpoint `https://www.arbeithnow.com/api/job-board-api` returns JSON of recent listings with no auth required [22†L68-L77] . This is a key feed, covering many companies’ official postings.
- **RemoteOK API:** RemoteOK provides a free JSON API of all remote job listings (with link attribution request) [16†L28-L31] . Example: `http://remoteok.com/api` returns an array of job objects (id, company, position, description, location, tags, apply URL, etc.). This can be polled directly without login.
- **We Work Remotely (RSS):** WWR offers public RSS feeds (e.g. [weworkremotely.com/remote-jobs.rss](https://www.weworkremotely.com/remote-jobs.rss)) for each category. We can fetch and parse RSS to get new postings. RSS is explicitly public.
- **USAJOBS API:** If interested in US government jobs, USAJOBS provides a free REST API (Registration required) [30†L123-L131] . We would need to sign up for a free key. (Optional for remote worldwide focus; note strict TOS [30†L108-L117] .)
- **Others:** We can add more aggregators as needed (e.g. [Arbeithnow’s blog lists many job APIs](#) [22†L108-L117] like Adzuna, Jobs2Careers, etc.). However, many require API keys or are region-specific. We’ll start with the above free sources.

2.2 Company Career Pages & ATS

Many tech companies use **cloud-based ATS** (Applicant Tracking Systems) which publish public job feeds or APIs. We can leverage these official endpoints:

- **Greenhouse Job Board API:** Companies on Greenhouse (like Cisco, Adobe, etc.) have a public JSON feed at `https://boards-api.greenhouse.io/v1/boards/{company_token}/jobs`. No auth is needed for GET (the “board token” is usually the subdomain in their jobs URL `[5†L164-L172]`). Including `?content=true` yields full job descriptions `[5†L99-L108]`.
- **Lever Postings API (OAuth):** Lever provides a JSON API (`https://api.lever.co/v1/postings/{company}`) that returns all open jobs. It requires an API key or OAuth. If the user has connections/employers on Lever, they can generate a key (Kept in GitHub Secrets). (Caveat: authentication needed; if avoided, we can skip Lever unless credentials are available.)
- **SmartRecruiters:** Companies on SmartRecruiters may have a public API (API tokens needed, see [SmartRecruiters Developer](#)).
- **Teamtailor:** Teamtailor jobs sometimes have an XML/JSON feed. (Their docs mention an always-included XML feed for partners `[13†L33-L41]`.) We can attempt to fetch a company’s Teamtailor career page and parse the embedded JSON (similar to how GraphQL feeds can be scraped). (No official public unauthenticated API, but many Teamtailor sites embed job data.)
- **Other ATS (Workday, Jobvite, etc.):** Some companies (e.g. Microsoft, IBM) use Workday/Jobvite with JSON interfaces (often hidden). We could list common patterns (e.g. Workday has structured URLs). Scraping Workday HTML is brittle, so we may rely on aggregators instead.

Important: We only connect to these if allowed. Greenhouse is fully public (no key needed) `[5†L66-L74]`. If we automate application submission (beyond scope), those endpoints require auth (not in our free plan). We only use read-only feeds.

3. Tools & Technology Inventory

We deliberately choose **free, open-source** tools. Below is an inventory with pros/cons.

3.1 Crawling & Automation Tools

Tool / Library	License	Use-Case	Notes
GitHub Actions	Proprietary (CI)	Scheduler and tasks execution	Free for public repos (min. 2000 min/month). Can trigger on schedule or webhooks. Easy to use, integrates with GitHub. (Limit: Cron min interval ~15m reliably.)
n8n (self-hosted)	Source-available (Fair-code) <code>[32†L115-L123]</code>	Workflow automation (low-code)	Node-based drag&drop flows. Powerful, many integrations. Requires technical setup. Not <i>fully open</i> (FairCode), but free to self-host. Good for complex workflows or conditional logic. +Extensible via JS. See n8n docs .

Tool / Library	License	Use-Case	Notes
ActivePieces	MIT (Open-source) [32†L98-L100]	Workflow automation (low-code)	Modern step-based builder. Fully MIT-licensed OSS, easy Docker install. Built-in AI connectors. Good for non-developers. Very fast start-up. (Docs: activepieces.com)
Huginn	MIT (OSS) [38†L52-L60]	Agent-based automation (like IFTTT)	Ruby on Rails scheduler. You create “Agents” that fetch feeds, scrape HTML, send actions. Open source for 10+ years [38†L52-L60] . Great at RSS scraping and site-watching. More developer-centric. Not graphically modern. Supports many triggers/actions (Webhooks, email, RSS).
Cron (Linux)	OS-native	Simple scheduling via server	If self-hosting, can use cron jobs or systemd timers. Not needed if GH Actions covers scheduling.
Cheerio (Node)	MIT	HTML parsing in Node	Fast DOM parsing (jQuery-like) for HTML pages. Good if any needed scraping.
rss-parser (JS/Python)	MIT	Parse RSS/Atom feeds	E.g. <code>rss-parser</code> npm or Python <code>feedparser</code> . For WeWorkRemotely etc.
Requests/urllib (Python)	Apache 2	HTTP client libraries	For any custom fetch.
Playwright/Puppeteer	MIT	Browser automation	<i>Not recommended:</i> it simulates browser to scrape sites (e.g. LinkedIn search). Violates LinkedIn TOS and can get blocked. Use <i>only</i> if no API exists and you accept risk. Not needed here.

Key Points:

- We **recommend GitHub Actions** as the main orchestrator, since it’s free for public repos and integrates easily with Supabase and CLI tools. We can schedule workflows (max every 15 minutes) and run Node/Python scripts.
- **n8n, ActivePieces, Huginn** are alternatives if one prefers a visual builder or need event-driven triggers. They are all self-hostable (no cost). *ActivePieces* is fully OSS (MIT) with modern UI [\[32†L98-L100\]](#) . *n8n* is powerful but uses a proprietary license (Fair-code) [\[32†L115-L123\]](#) and can be more complex. *Huginn* is older but very flexible for RSS/HTML tasks [\[38†L52-L60\]](#) .
- **No Paid Services:** We avoid Zapier, IFTTT cloud, SeleniumGrid, or any non-free API (LinkedIn API, etc.).
- For **crawling**, we rely on official feeds (JSON, RSS). If needed, basic HTTP + HTML parsing (Cheerio) can fetch simple job listings (like a tech blog or HN thread). But heavy HTML scraping (like Indeed or LinkedIn) is out of scope.

3.2 AI Models & Libraries

The resume tailoring and scoring use free AI models:

- **HuggingFace Transformers:** Many pretrained models can run locally (small ones like `t5-small`, `gpt2`, etc.). We use **Nakamoto's T5 model** on HuggingFace, fine-tuned to generate resume JSON from a job description `【28†L203-L212】`. We can download this model with `transformers` in Python or via HuggingFace's inference API (free tier).
- **Sentence Embeddings:** For job-scoring by skill match, libraries like [SentenceTransformers](#) (e.g. `all-MiniLM`) can compute similarity between job text and your resume. These are Python libs (MIT/Apache) and free to use.
- **LangChain/OpenAI:** We avoid paid APIs like OpenAI for production. If needed for one-time setup, OpenAI's free trial (for initial tests) is optional but not required. Instead, we focus on **local LLMs**:
- **LLaMA-2 (Meta)** 7B/13B models (open-weight for research) can run via [llama.cpp](#) on a sufficiently large runner (GH Actions' ubuntu has ~60GB, 8-core; it may handle 7B model with quantization).
- **Mistral, Bloom, GPT-J:** also OSS but much larger; likely too big for simple GH setup.
- For **summarization or rewriting**, smaller models suffice (e.g. `t5-small` or `facebook/bart-large-cnn` for summarizing JDs or CV text).
- **LaTeX Template:** We will embed the AI-generated content into a LaTeX resume template. (Templates like *moderncv* or *Awesome-CV* are MIT/BSD and free). The final PDF is generated with `pdflatex` in GH Actions or in Next.js using `latex.js`. A sample LaTeX skeleton will be provided.
- **Comparison of AI Options:**

Model/Service	Free?	Notes
Nakamoto's T5-resume (HF) <code>【28†L203-L212】</code>	✓ (OSS)	Generates JSON CV from JD. Good for structure.
MiniLM / SBERT	✓ (Apache 2)	Embedding-based matching. Low CPU.
LLaMA-2 7B (Meta licence)	✓	Large, can generate natural text if needed.
BART / T5	✓	Summarize or rewrite JDs/CVs.
OpenAI/GPT (API)	✗ (paid)	Not free, avoid in production.
Claude (API)	✗ (paid)	Not free.
Local LLM (GPT4All)	✓ (MIT)	Many local models exist (gpt4all, etc.), but quality is limited.

We choose open-source models (T5, SBERT, etc.) to avoid costs. If running on GH Actions, careful with RAM/CPU (stick to 7B or smaller models). The references above include [Nakamoto's T5 model](#) which is explicitly designed for resume tailoring.

4. Architecture & Data Flow

Below is a high-level architecture and data flow (Mermaid diagram):

```
graph LR
    subgraph Scheduler
        A[GitHub Actions Cron]
    end
    subgraph Source_Fetch [Source Fetch]
        B[Job Crawler Node.js/Python]
        C1[Arbeitsnow API]
        C2[RemoteOK JSON]
    end
    A --> B
    B --> C1
    B --> C2
```


pdf_ur1 | TEXT | URL to PDF file (stored in Supabase storage) | | ats_score | FLOAT |
Optional: ATS compatibility score | | created_at | TIMESTAMP | When generated |

- **applications** (applications tracking)

Column	Type	Description	id (PK)
UUID			
job_id	FK → jobs.id	applied_at	result
e.g. "interview", "offer", "rejected"		notes	

(User can store notes)

- **companies** (if needed for career page URLs)

Column	Type	Description	name (PK)	TEXT	Company
career_url	TEXT	Their jobs page (if known)	last_scan	TIMESTAMP	For monitoring

The **Supabase schema** is defined via SQL migrations (see Implementation). We enable Row Level Security (RLS) so that only the user or webhook (via anon key) can insert jobs. All data access goes through Supabase's PostgREST API or its JavaScript client.

Architecture Notes

- **Scalability:** The system is light-weight. Job polling every 15 minutes on free tiers is sufficient. Supabase free includes 500k row writes/month, which is ample if we dedupe. Next.js dashboard can be hosted on Vercel free or even on Supabase Edge (via its new Edge Functions).
- **Security:** Secrets (Supabase URL/Key, any API tokens) are stored as GitHub Actions secrets. Supabase auth can restrict access to only our user. We never store sensitive personal data in logs. All external fetches respect robots.txt and API TOS.
- **Mermaid Diagrams:** The above flowchart, and a data flow chart, are embedded as Mermaid (no external image needed).

5. GitHub Actions and Workflows

We will use GitHub Actions for scheduling and running the pipeline. Example workflow (file: `.github/workflows/job-hunter.yml`):

```
name: AI Job Hunter

on:
  schedule:
    # Every 15 minutes
    - cron: '*/*15 * * * *'
  workflow_dispatch: # manual trigger option

jobs:
  scan_and_process:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Setup Node (or Python) environment
      - name: Setup Node.js
```

```

    uses: actions/setup-node@v4
    with:
      node-version: '18'

  # Install dependencies (if Node.js)
  - name: Install dependencies
    run: npm ci

  # Fetch jobs from feeds
  - name: Run Job Crawler
    run: |
      node scripts/fetch_jobs.js \
        --arbeitnow="https://www.arbeitnow.com/api/job-board-api" \
        --remoteok="http://remoteok.com/api" \
        --weworkrssh="https://weworkremotely.com/remote-jobs.rss"

  # Deduplicate & score
  - name: Run Dedup & Scoring
    run: |
      node scripts/process_jobs.js

  # Generate resumes for top jobs
  - name: Generate Resumes with AI
    uses: docker://python:3.11-slim
    with:
      args: >
        pip install -r resume_requirements.txt &&
        python scripts/generate_resumes.py

  # Push changes to Supabase (using supabase CLI)
  - name: Push to Supabase DB
    env:
      SUPABASE_URL: ${ secrets.SUPABASE_URL }
      SUPABASE_SERVICE_KEY: ${ secrets.SUPABASE_KEY }
    run: |
      supabase db diff --local ./supabase/migrations || true
      supabase db push --accept-unauthorized

```

This YAML schedules the job every 15 minutes (`cron: '*/*15 * * * *`), checks out code, installs Node, runs scripts to fetch and process jobs, runs a Python step (for AI resume generation), and uses the `supabase` CLI to apply any DB schema changes. All secrets (like `SUPABASE_URL`) are kept safe.

Key points:

- **Scheduling:** The minimum GitHub Actions cron interval is 5 minutes, but we choose 15m to be polite. Even 15m yields ~96 runs/day, which is OK under GH Actions free limits.
- **Modularity:** Each step is separate. We could also split into multiple jobs (e.g. one for crawling, one for AI), but a single job with steps is simpler.
- **Self-Hosting alternatives:** Instead of Actions, one could use n8n or ActivePieces to trigger a webhook that runs code on a server (e.g. a free VM or GH self-host runner). But Actions is sufficient and simpler (no extra infra needed).

6. Crawlers, Deduplication & AI Pipeline

6.1 Crawlers / Fetchers

For each data source, we implement a fetcher:

- **Arbeitsnow:** Use `node-fetch` or `axios` to GET the JSON endpoint. Iterate through `data[]`, extracting fields (`company_name`, `title`, `description`, `location`, `remote`, `url` etc. from each item). Use `date` or `created_at` to skip old jobs.
- **RemoteOK:** Similar GET on `http://remoteok.com/api`. The JSON is an array of job objects (skip the first element which is metadata). Each job has `id`, `company`, `position`, `location`, `description`, `apply_url`, etc.
- **WeWorkRemotely:** Fetch RSS (e.g. with `rss-parser` in Node or `feedparser` in Python). Each `<item>` has title, link, description, pubDate. Convert to job object.
- **Greenhouse (example):** If a `company_token` is configured (e.g. "stripe"), fetch `https://boards-api.greenhouse.io/v1/boards/stripe/jobs?content=true`. This returns `{ jobs: [ ... ] }`. Loop through jobs.
- **Rate Limiting:** We space requests. Since Actions run every 15m, even fetching 3–5 sources is light. Respect each API's guidelines (e.g. RemoteOK asks to link back, which we do on the dashboard).

Each fetcher produces a list of jobs in a unified format. We assign each job a composite unique key (e.g. source + external_id or slug).

6.2 Deduplication

We store a **fingerprint** for each job to avoid repeats. Approaches:

- **Persistent DB check:** Before inserting a job to `jobs` table, check if an entry with the same `external_id` and `source` already exists. If so, skip. (Mark any updated postings, if needed, by checking a changed timestamp.)
- **Fuzzy matching:** For safety, also compare new job's (title + company) against recent entries in DB (e.g. last week). If Levenshtein distance is small, treat as duplicate. (Open-source libs: `fuse.js`, `rapidfuzz` in Python.)
- **Content hash:** Hash the job description text (md5) and skip if identical description exists.

Deduplication ensures the pipeline processes each job only once.

6.3 Scoring & Ranking

We compute a **match score** for each job vs. our profile. Example algorithm:

1. **Skills match:** Extract required skills from the JD (via keyword matching or simple NLP). Compare against our skills list. Compute percentage match.
2. **Experience level:** If we want Senior roles, detect if JD says "Senior" or "Lead". If it's junior, penalize score.
3. **Location/Remote:** Compare location vs. preferences (e.g. prefer Bangalore or Remote).
4. **Company preference:** (Optionally) weight some companies higher if we like them.

5. **Keyword embedding:** Use an embedding model (e.g. `all-MiniLM`) to vectorize the job description and vectorize our resume/bio. Compute cosine similarity.
6. **ATS-friendliness:** If known, check for some resume keywords.

We can implement this as a simple formula, or even use a small LLM to output a score. But to keep it free, a custom script is best. For example:

```
// Pseudocode for scoring
score = 0;
if (job.level == desired_level) score += 30;
score += min(30, 10 * matchingSkillsCount);
if (job.remote && preferRemote) score += 20;
if (distance < 50km) score += 10;
score += embeddingSimilarity * 40; // scale cosine similarity to 0-40
// Cap at 100
job.score = Math.min(score, 100);
```

We store this `score` in `jobs.score`. Higher-scoring jobs will be surfaced on the dashboard.

6.4 AI Resume Generation

For each **shortlisted** job (score above threshold, e.g. 70%), we generate a tailored resume:

1. **Prepare prompt:** Combine the job description and possibly our master resume. For example:

Generate a JSON resume for the following job:
[Job Title] at [Company]. [Full job description text].
Focus on including relevant skills (AWS, Kubernetes, etc.), achievements, and using professional tone.

1. **Run model:** Use the `nakamoto-yama/t5-resume-generation` model [28†L203-L212] via HuggingFace Transformers. This model outputs a JSON-like string (with special tokens `LB>` / `RB>` for braces). Example code (Python):

```
from transformers import T5Tokenizer, T5ForConditionalGeneration
model_path = "nakamoto-yama/t5-resume-generation"
tokenizer = T5Tokenizer.from_pretrained("google/t5-base")
model = T5ForConditionalGeneration.from_pretrained(model_path)
prompt = f"generate resume JSON for the following job: {job_title} at {company}. {job_description}"
inputs = tokenizer(prompt, return_tensors="pt", padding=True)
outputs = model.generate(**inputs, max_length=512)
resume_json = tokenizer.decode(outputs[0], skip_special_tokens=True)
resume_json = resume_json.replace("LB>", "{").replace("RB>", "}")
```

We parse this JSON into a Python dict.

1. **Merge with LaTeX template:** We have a LaTeX template file (for the job's role). We fill placeholders like `{{NAME}}`, `{{POSITION}}`, and bullet lists (skills, experience points) from the JSON. This can be done with a templating engine (e.g. Jinja2 for `.tex`).
2. **Output PDF:** Run `pdflatex` on the filled `.tex` to produce a PDF (stored in Supabase Storage or as a BLOB). Also save DOCX if needed (Pandoc can convert LaTeX to DOCX, or generate directly using a library like `docx`). The file URLs are stored in `resumes.pdf_url`.
3. **ATS-check (optional):** We could run an open-source ATS analyzer (e.g. `resumake` script or Python library) to score how ATS-friendly the resume is, and store that in `ats_score`.

All models and libraries used are open-source (HuggingFace, transformers, PyTorch). No proprietary API calls are needed. [Nakarmoto's model](#) was specifically trained for this resume task, making it ideal.

6.5 Deduplication & Storage (reprise)

After generation, we update the `jobs` and `resumes` tables in Supabase. Each job (with its score) is inserted if new, and if a tailored resume was made, a row in `resumes` is inserted with links to the generated PDF.

Security: We only allow insertion via the Action's `SERVICE_KEY`, not publicly.

7. Implementation Guidelines

This section sketches the **step-by-step setup**, with references to OSS docs and example code.

7.1 Infrastructure Setup

1. **Create GitHub Repo (public).** Initialize with a license (MIT) and README. Set up the file structure:

```
.github/  
  workflows/job-hunter.yml  
src/  
  scripts/  
    fetch_jobs.js  
    process_jobs.js  
    generate_resumes.py  
supabase/  
  migrations/  
templates/  
  resume_template.tex
```

1. **Supabase Project:** Sign up for Supabase (free). Create a new project (Postgres database). Copy the `SUPABASE_URL` and `SUPABASE_SERVICE_KEY` into GitHub Secrets (to use in Actions). Initialize Supabase CLI locally: `supabase init`.

2. **DB Schema:** In `supabase/migrations/`, create SQL migration files. Example using `supabase` CLI:

```
supabase migration new init_tables
```

Then edit the migration SQL (`supabase/migrations/0001_init_tables.sql`):

```
CREATE TABLE jobs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source TEXT NOT NULL,
  external_id TEXT,
  title TEXT NOT NULL,
  company TEXT NOT NULL,
  location TEXT,
  remote BOOL,
  salary_min INT,
  salary_max INT,
  description TEXT,
  apply_url TEXT,
  posted_at TIMESTAMP,
  score INT,
  status TEXT DEFAULT 'new',
  created_at TIMESTAMP DEFAULT now(),
  UNIQUE(source, external_id)
);
CREATE TABLE resumes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id UUID REFERENCES jobs(id) ON DELETE CASCADE,
  version TEXT,
  content JSONB,
  pdf_url TEXT,
  ats_score FLOAT,
  created_at TIMESTAMP DEFAULT now()
);
CREATE TABLE applications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id UUID REFERENCES jobs(id) ON DELETE CASCADE,
  applied_at TIMESTAMP,
  result TEXT,
  notes TEXT
);
CREATE TABLE companies (
  name TEXT PRIMARY KEY,
  career_url TEXT,
  last_scan TIMESTAMP
);
```

Save, and push migrations:

```
supabase db push
```

This creates the tables in the Supabase database [\[44†L439-L447\]](#) .

1. **Row-Level Security:** (optional) Enable RLS if needed:

```
ALTER TABLE jobs ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "insert_jobs" ON jobs FOR INSERT TO authenticated USING (true);
```

So that only authenticated API calls (via service role) can insert.

7.2 Crawler Code

- **fetch_jobs.js (Node):** This script fetches feeds and writes to a JSON file or directly to DB.
Pseudocode outline:

```
const axios = require('axios');  
const Parser = require('rss-parser');  
async function fetch() {  
  let jobs = [];  
  // Arbeitnow  
  let res = await axios.get(process.env.ARBEIT_URL);  
  jobs.push(...res.data.data.map(item => ({  
    source: 'arbeitnow',  
    external_id: item.slug,  
    title: item.title,  
    company: item.company_name,  
    description: item.description,  
    location: item.location,  
    remote: item.remote,  
    apply_url: item.url,  
    posted_at: new Date(item.created_at * 1000)  
  })));  
  // RemoteOK  
  res = await axios.get(process.env.REMOTEOK_URL);  
  let arr = res.data.slice(1); // skip first metadata element  
  arr.forEach(job => jobs.push({  
    source: 'remoteok',  
    external_id: job.id,  
    title: job.position,  
    company: job.company,  
    description: job.description,  
    location: job.location,  
    remote: true,  
    apply_url: job.apply_url,  
    posted_at: new Date(job.epoch * 1000)  
  }));  
  // WeWorkRemotely RSS  
  let parser = new Parser();
```

```

let feed = await parser.parseURL(process.env.WEWORK_RSS);
feed.items.forEach(item => jobs.push({
  source: 'weworkremotely',
  external_id: item.link,
  title: item.title,
  company: '', location: '',
  description: item.content,
  remote: true,
  apply_url: item.link,
  posted_at: new Date(item.pubDate)
}));
// TODO: Insert into DB via Supabase (see step 7.4)
}
fetch();

```

- **process_jobs.js (Node):** This script deduplicates and scores, then inserts into Supabase via its REST API:

```

const { createClient } = require('@supabase/supabase-js');
const supabase = createClient(process.env.SUPABASE_URL,
process.env.SUPABASE_KEY);

async function run() {
  let newJobs = require('../data/jobs.json'); // or from fetch
  for (let job of newJobs) {
    // Check duplicate
    let { data: existing } = await supabase
      .from('jobs')
      .select('id')
      .eq('source', job.source)
      .eq('external_id', job.external_id);
    if (existing && existing.length > 0) continue; // skip
    // Score job
    job.score = computeScore(job); // define this function
    // Insert job
    await supabase.from('jobs').insert(job);
  }
}
run();

```

7.3 AI Resume Script

- **generate_resumes.py:** (Python with Transformers)

```

import os, json
import psycopg2
from transformers import T5Tokenizer, T5ForConditionalGeneration

# Connect to Supabase Postgres
conn = psycopg2.connect(os.environ['DATABASE_URL'], sslmode='require')

```

```

cur = conn.cursor()
# Load model
tokenizer = T5Tokenizer.from_pretrained("google/t5-base")
model = T5ForConditionalGeneration.from_pretrained("nakamoto-yama/t5-resume-generation")

# Fetch top unprocessed jobs
cur.execute("SELECT id, title, company, description FROM jobs WHERE status='shortlisted'")
jobs = cur.fetchall()
for job_id, title, company, desc in jobs:
    prompt = f"generate resume JSON for the following job: {title} at {company}. {desc}"
    inputs = tokenizer(prompt, return_tensors="pt", padding=True)
    outputs = model.generate(*inputs, max_length=512)
    resume_json = tokenizer.decode(outputs[0], skip_special_tokens=True)
    resume_json = resume_json.replace("LB>", "{").replace("RB>", "}")
    resume = json.loads(resume_json)
    # Merge with LaTeX template
    tex = render_latex(resume, title, company) # user-defined function
    tex_file = f"output/{job_id}.tex"
    with open(tex_file, 'w') as f: f.write(tex)
    # Compile LaTeX to PDF (via subprocess)
    os.system(f"pdflatex -output-directory=output {tex_file}")
    pdf_url = upload_to_supabase_storage(f"output/{job_id}.pdf") # user-defined
    # Insert resume record

cur.execute("INSERT INTO resumes (job_id, version, content, pdf_url) VALUES (%s,%s,%s,%s)",
            (job_id, "tailored_v1", json.dumps(resume), pdf_url))
    # Update job status
cur.execute("UPDATE jobs SET status='resume_generated' WHERE id=%s", (job_id,))
conn.commit()
conn.close()

```

Notes: `render_latex` would load a Jinja2 `.tex` template (see next section) and fill it with `resume` fields (bullet points, skills, etc.). `upload_to_supabase_storage` would use Supabase Python client to upload the PDF to the Storage bucket and return the URL. All of this uses only free packages (`transformers`, `psycopg2`, `jinja2`).

7.4 LaTeX Resume Templates

We prepare a generic LaTeX resume template with placeholders for name, contact, skills, experience, etc. For example, using the `moderncv` class:

```

% templates/resume_template.tex
\documentclass[11pt,a4paper]{moderncv}
\moderncvstyle{classic}

```

```

\moderncvcolor{blue}
\usepackage[utf8]{inputenc}

\name{John}{Doe}
\title{\VAR{job_title}}
\address{Bengaluru, India}{}{}
\phone[mobile]{+91~123~456~7890}
\email{example@gmail.com}

\begin{document}
\makecvtitle

\section{Experience}
\BEGIN{itemize}
\FOR skill in resume["skills"]:
  \item \VAR{skill}
\END{itemize}

\section{Education}
\textbf{M.Sc. in Computer Science} \hfill 2014--2016\\
University of Bengaluru

\section{Projects}
\BEGIN{itemize}
\FOR project in resume["projects"]:
  \item \textbf{\VAR{project["title"]}}: \VAR{project["desc"]}
\END{itemize}

\section{Skills}
\VAR{"", ".join(resume["skills"])}

\end{document}

```

This template uses a templating syntax (e.g. Jinja-like) to fill in the `title`, `skills`, `projects` from the JSON resume. (In practice, we'd use actual Jinja2 tags or another templating system.)

As **initial examples**, for a “Platform Engineer” role, we might output:

```

\name{Alice}{Bhardwaj}
\title{Senior Platform Engineer}

\section{Experience}
\begin{itemize}
  \item Designed CI/CD pipelines with Jenkins and GitHub Actions for microservices (Docker/Kubernetes) deployments.
  \item Managed AWS infrastructure (EC2, S3, VPC) and automated Terraform scripts to reduce manual overhead.
  \item Led a team of 3 DevOps engineers to implement IaC (Terraform, CloudFormation) at scale.
\end{itemize}
\end{document}

```



```
\section{Education}
```

B.Tech in Computer Science, IIT Delhi, 2014-2018

```
\section{Skills}
```

AWS, Kubernetes, Docker, Terraform, Python, Linux, Git, Jenkins

```
\end{document}
```

For a “Backend Developer” role, the LaTeX template would be similar but with a different title. (We can maintain one template file and just change the `\title{}` and bullet points.) The key is that all content is filled via the AI outputs, so no manual editing is needed.

7.5 Next.js Dashboard

For the UI, we can spin up a simple Next.js app (free on Vercel or Netlify) that queries Supabase. Libraries:

- `@supabase/supabase-js` for DB access
- A React page showing a table of `jobs` (title, company, score, status) with an “Apply” button linking to `apply_url` and resume link.
- Ability to click “Applied” which updates the `jobs.status` via Supabase (closing the loop).
- Styling can use any free component library (Tailwind, shadcn, etc.).

This is mostly standard full-stack React coding (see [ResumeLM’s stack in their README](#), which uses Next.js and Supabase).

7.6 Testing & Monitoring

- **Testing:** Write unit tests for crawlers (e.g. using Jest or pytest) to ensure parsing works. GitHub Actions can also run tests on PRs.
- **Monitoring:** Use Actions logs, or set up Slack/Discord notifications for workflow failures (via a GitHub webhook or action). Supabase also provides logs and metrics (especially on the free tier limited).

All code should be linted and formatted (e.g. `eslint` for JS, `black` for Python). The example above (ResumeLM [44†L439-L447]) shows a production-grade stack, confirming these choices.

8. Operations & Future Plans

8.1 Scaling & Performance

- **Load:** The system is lightweight. Even scanning dozens of sources every 15m is trivial. Supabase free tier supports 500k monthly inserts (ample).
- **Scaling:** If more users or companies are added, we may move to higher-tier actions or host our own runner. We might batch jobs less frequently if hitting GitHub minutes limits.
- **Optimizations:** Cache API responses, use incremental updates (Arbeitnow supports paging by date). Clean up old jobs periodically (e.g. mark status “expired” if past a date).

8.2 Learning Loop

We support a feedback loop for personalization:

- Whenever a job is **applied/interviewed/rejected** in the dashboard, we log that. Over time, we can train a small ML model to re-weight the scoring features (e.g. if all "rejected" jobs were remote only, maybe down-weight remote preference).
- For simplicity, start with manual adjustments (e.g. change required skills list). Later, use logistic regression or an online learning model on the (features → decision) data.

This does not require new infra; just code in our scoring step to read past decisions from the `jobs` table and adapt weights (e.g. boost companies where we got interviews).

8.3 Privacy & Compliance

- **Data Handling:** Only store minimal personal info (name in resume JSON). Do not expose private details. The system is for personal use, so no PII sharing.
- **Site TOS:** We strictly avoid unauthorized scraping. Only official feeds/APIs and public RSS are used. The [Huginn article](#) even warns about TOS-compliance. We encourage respecting robots.txt and rate limits.
- **GDPR/ATS:** We generate resumes and track applications in Supabase. This is personal data (your resume). Supabase is GDPR-compliant. We should secure the dashboard behind a login (or at least not expose it publicly if the resume has full details). Alternatively, just use it personally.

8.4 Maintenance and Upgrades

- **Open-Source Models:** Keep models up-to-date. New better open models may emerge (e.g. Llama-3). We can upgrade the Transformers code as needed.
- **Tool Comparison:** The automation tools (n8n, ActivePieces) are evolving. We chose GH Actions, but if in future we need more triggers or non-code users, we can add a self-hosted n8n or ActivePieces instance (both Docker images are available).

Comparison Table (summary):

Tool	License	Hosted/ OSS	Best For	Notes
GitHub Actions	CI (free)	Proprietary	Scheduling & running scripts	Free for public repos, 2k mins/mo [44†L439-L447] . Integrates with Git/GitHub.
n8n (OSS)	Fair Code [32†L115-L123]	Self-host	Complex workflows, APIs	Node-based flows, requires dev skills. Big community.
ActivePieces	MIT (OSS) [32†L98-L100]	Self-host	Easy flow building	Step-by-step UI, built-in AI, completely open source.
Huginn	MIT (OSS) [38†L52-L60]	Self-host	RSS/HTML monitoring	Event-driven "agents", great for RSS. Less modern UI.

Tool	License	Hosted/ OSS	Best For	Notes
Playwright/ Puppeteer	MIT	Self-host	Browser automation (legacy)	Not for LinkedIn (TOS violation risk).

(Sources: tool docs and the Activepieces blog [\[32†L98-L100\]](#) [\[38†L52-L60\]](#) .)

9. LaTeX Resume Templates (Examples)

Below are **sample LaTeX resumes** for different roles. These are *initial skeletons* to be filled by the AI.

9.1 Senior Platform Engineer

```

\documentclass[11pt,a4paper]{moderncv}
\moderncvstyle{classic}
\moderncvcolor{blue}
\name{Amal}{Singh}
\title{Senior Platform Engineer}
\address{Bengaluru, India}{}{}
\phone[mobile]{+91~987~654~3210}
\email{amal.singh@example.com}

\begin{document}
\makecvtitle

\section{Experience}
\begin{itemize}
\item Designed and managed scalable infrastructure on AWS (EC2, VPC, RDS), supporting 50k+ daily users.
\item Automated deployments with Jenkins and GitHub Actions; implemented zero-downtime Kubernetes upgrades.
\item Led team of DevOps engineers to build CI/CD pipelines (Docker, Terraform, Helm) for microservices.
\end{itemize}

\section{Education}
\textbf{M.Tech in Computer Science} \hfill 2018--2020\\
Indian Institute of Technology, Delhi

\section{Skills}
AWS, Kubernetes, Docker, Jenkins, Terraform, Python, Linux, CI/CD, Monitoring (Prometheus, Grafana)

\end{document}

```

9.2 Backend Developer

```
\documentclass[11pt,a4paper]{moderncv}
\moderncvstyle{classic}
\moderncvcolor{green}
\name{Priya}{Sharma}
\title{Backend Developer}
\address{Bengaluru, India}{}{}
\phone[mobile]{+91~123~456~7890}
\email{priya.sharma@example.com}

\begin{document}
\makecvtitle

\section{Experience}
\begin{itemize}
  \item Developed RESTful APIs in Node.js/Express for e-commerce platform,
  processing 10k+ transactions/day.
  \item Integrated Redis caching and database sharding to improve response
  times by 40%.
  \item Collaborated with frontend team to design GraphQL interfaces and
  optimize performance.
\end{itemize}

\section{Education}
\textbf{B.E. in Computer Science} \hfill 2015--2019\\
Vishwakarma Institute of Technology, Pune

\section{Skills}
Node.js, Express, PostgreSQL, MongoDB, Redis, GraphQL, Docker, Linux, AWS
(Lambda, S3)

\end{document}
```

9.3 DevOps Engineer

```
\documentclass[11pt,a4paper]{moderncv}
\moderncvstyle{casual}
\moderncvcolor{purple}
\name{Rahul}{Verma}
\title{DevOps Engineer}
\address{Bengaluru, India}{}{}
\phone[mobile]{+91~555~123~4567}
\email{rahul.verma@example.com}

\begin{document}
\makecvtitle

\section{Experience}
```

```

\begin{itemize}
  \item Built and maintained CI/CD pipelines using GitLab CI and Kubernetes.
  \item Containerized legacy applications with Docker, reducing environment inconsistencies.

\item Configured monitoring/alerting (Prometheus + Alertmanager) to improve uptime from 97\% to 99.9\%.
\end{itemize}

\section{Education}
\textbf{M.Sc. in IT} \hfill 2014--2016\\
University of Hyderabad

\section{Skills}
Docker, Kubernetes, GitLab CI, Terraform, AWS (CloudFormation), Linux, Shell Scripting, Networking

\end{document}

```

These templates use free LaTeX classes (*moderncv*, *casual* style) and demonstrate sections. In practice, our AI pipeline would generate the bullet points (the `itemize` content) and fill them in. The styling is customizable (colors, fonts). (For more polish, one could use templates like *Awesome-CV*, but that's just CSS changes.)

The **AI approach**: Instead of writing from scratch, the T5 model will output JSON like `{"skills": ["AWS", "Kubernetes", ...], "projects": [...], "experience": [...]}`. We then inject those into the LaTeX as shown.

References

- Greenhouse Job Board API [\[5†L164-L172\]](#) : Official public JSON for jobs (no auth needed).
- Arbeitnow Free Job Board API [\[22†L68-L77\]](#) : Aggregates jobs from many ATS (no key required).
- RemoteOK API/JSON Feed [\[16†L28-L31\]](#) : Remote jobs (public API, follow attribution).
- n8n vs ActivePieces Blog [\[32†L98-L100\]](#) [\[32†L115-L123\]](#) : Comparison of workflow tools and licenses.
- Huginn Automation (DEV blog) [\[38†L52-L60\]](#) : Open-source, self-hosted IFTTT-like agents.
- ResumeLM Stack [\[44†L439-L447\]](#) : Example of Next.js + Supabase + AI resume builder (uses supabase, indicates feasibility).
- Resume Maker (Advice) [\[45†L49-L57\]](#) : Suggests using AI to generate LaTeX code for resumes, illustrating approach.
- Supabase Overview [\[37†L15-L19\]](#) : Supabase is an open Postgres platform with instant APIs and Storage.

All external data is obtained via open/free channels. Models and libraries cited (Hugging Face, transformers, etc.) are open-source. This solution avoids any paid service and relies entirely on free resources.
